SWS3012_06B

# HHHHHH

Jiang Yongqian
Kong Anyuan
Su Yanfei
Zhu Zheyu

# Game Introduction

➢ **Game Type**

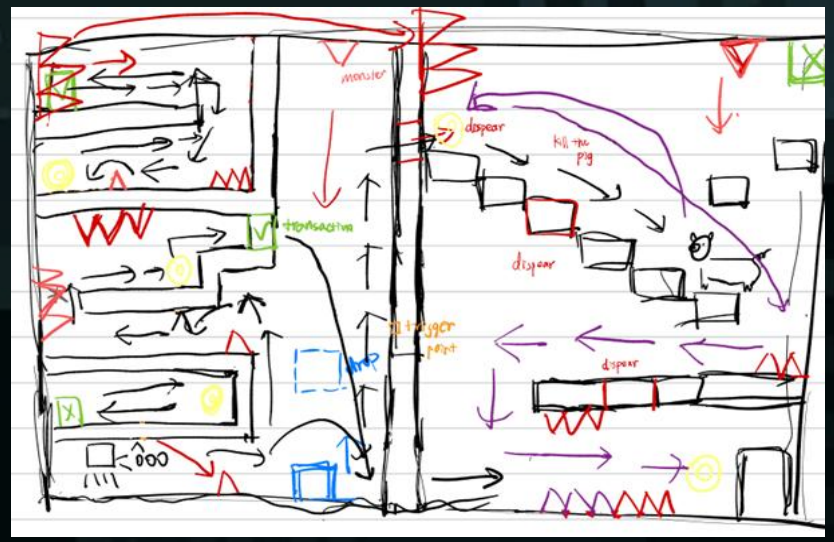
Our project name is UWANNA, which is a two-dimensional planar gravity operational game. It is designed based on arcade_2d. The game are equipped with unexpected ways to pass and trap types, with a variety of unique and creative designs.

➢ **Basic Design**

By pressing buttons, we can make player move forward and backward, jump and attack, and Players need to avoid various traps, use functional blocks to collect enough coins to earn points, and finally reach the finish line to end the game according to the designed route. If encountering the triangle, the game ends and returns to the starting point.

# Animation and Design

We have designed traps and functional blocks that trigger upon arrival, such as levitation super jump and teleportation. We have designed dynamic and death effects for player characters, adding a small monster and a pig as obstacles to add visual effects and fun to the game. And player activities are accompanied by corresponding sound effects such as death or getting score.



# Code

```
function check_collision(rect1, rect2) {
    const pos1 = query_position(rect1);
    const pos2 = query_position(rect2);
    return (math_abs(pos1[0] - pos2[0]) < 32) && (math_abs(pos1[1] - pos2[1]) < 32);
}
function check_collision1(rect1, rect2) {
    const pos1 = query_position(rect1);
    const pos2 = query_position(rect2);
    return (math_abs(pos1[0] - pos2[0]) < 27) && (math_abs(pos1[1] - pos2[1]) < 27);
}
```

```
//检查是否踩到尖刺
function check_gameover(rect1,rect2){
if (check_collision1(rect1, rect2)) {
    new_position=[20, ground_level - 20];
    nh=0;
    h1=0;
    score=0;
    n7=0;
    nw=0;
    w=0;
    move_monster=0;
    move=0;
    update_position(coin5,[150, ground_level - block_size * 4]);
    update_position(coin6,[130, ground_level - block_size * 10]);
    update_position(coin7,[40, ground_level - block_size * 25]);
    update_position(coin8,[400, ground_level - block_size * 20]);
    update_position(coin9,[470, ground_level - block_size*3]);

    update_position(triangle14, [210, ground_level - block_size * 8]);
    update_position(triangle15, [10, ground_level - block_size * 9]);
    update_position(triangle16, [10, ground_level - block_size * 10]);
```

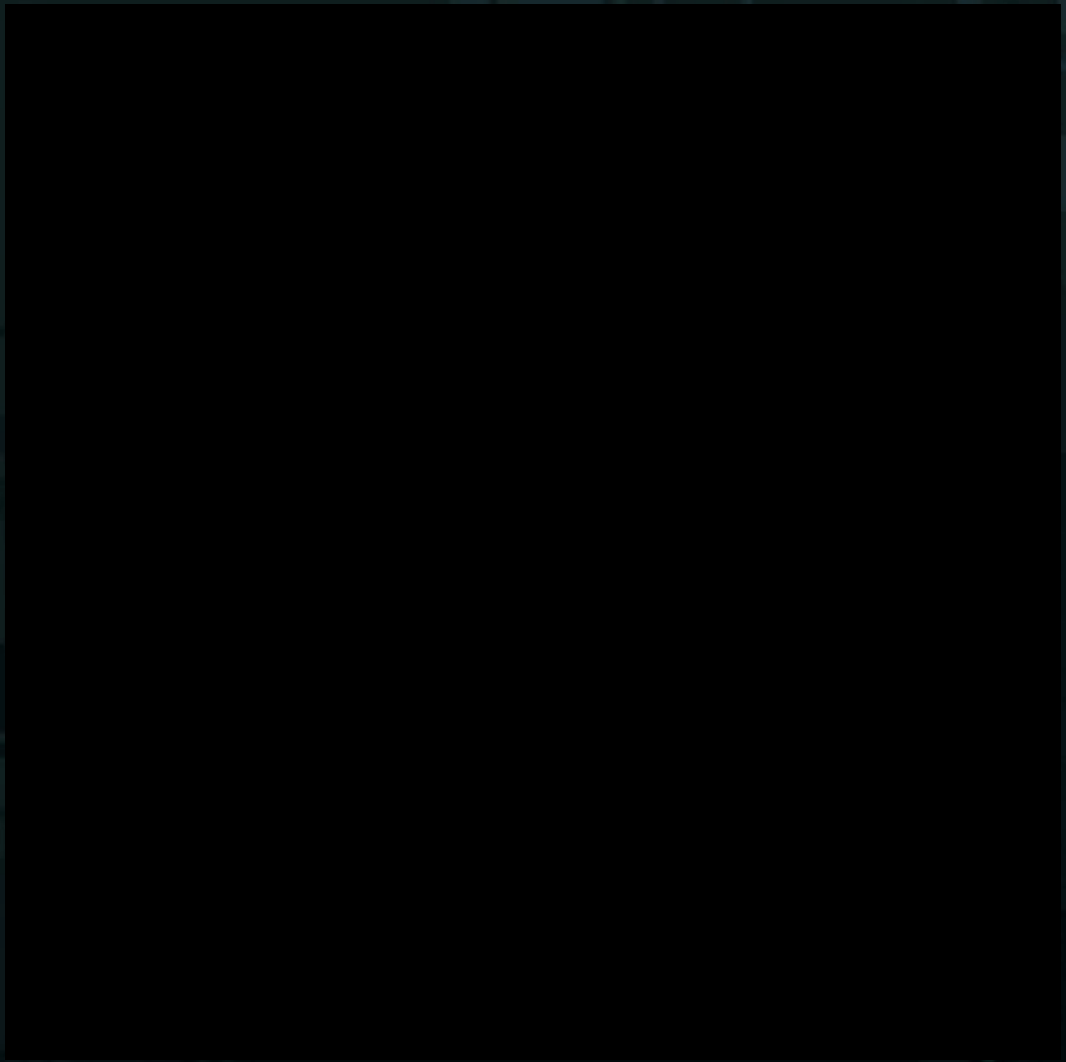
```
if (move_monster===1){
    if(move<380){
        move=move+0.8;
    }else if(move<450){
        move=move+1;
    }
    update_position(all_monster, [310, ground_level - block_size * 26 + move]);
}
```

```
if (walk2<2){
    all_monster=update_position(monster[0], query_position(all_monster));
    walk2=walk2+0.1;
    update_position(monster[1],[-100,-100] );
    update_position(monster[2],[-100,-100] );
    update_position(monster[3],[-100,-100] );
    update_position(monster[4],[-100,-100] );
}else if(walk2<3){
    all_monster=update_position(monster[1], query_position(all_monster));
    walk2=walk2+0.1;
    update_position(monster[0],[-100,-100] );
    update_position(monster[2],[-100,-100] );
    update_position(monster[3],[-100,-100] );
    update_position(monster[4],[-100,-100] );
}else if(walk2<4){
    all_monster=update_position(monster[2], query_position(all_monster));
    walk2=walk2+0.1;
    update_position(monster[0],[-100,-100] );
    update_position(monster[1],[-100,-100] );
    update_position(monster[3],[-100,-100] );
    update_position(monster[4],[-100,-100] );
} else if(walk2<5){
    all_monster=update_position(monster[1], query_position(all_monster));
    walk2=walk2+0.1;
    update_position(monster[0],[-100,-100] );
    update_position(monster[2],[-100,-100] );
    update_position(monster[3],[-100,-100] );
    update_position(monster[4],[-100,-100] );
} else if (walk2<6){
    all_monster=update_position(monster[0], query_position(all_monster));
    walk2=walk2+0.1;
    update_position(monster[1],[-100,-100] );
    update_position(monster[2],[-100,-100] );
```

```
    // check if reach the door
    if (check_collision3(player, door)){
            level = 2;
            for (let i=0; i<92; i=i+1){
                update_position(blockphoto[i], [-100, -100]);

            }
             update_position(back2,[300,40]);
            //on_ground=false;
      blocks2 = [
            update_position(blockphoto[0], [10, ground_level - block_size * 3]),
            update_position(blockphoto[1], [30, ground_level - block_size * 3]),
            update_position(blockphoto[2], [50, ground_level - block_size * 3]),
            update_position(blockphoto[3], [70, ground_level - block_size * 3]),
            update_position(blockphoto[4], [90, ground_level - block_size * 3]),
            update_position(blockphoto[5], [110, ground_level - block_size * 3]),
            update_position(blockphoto[6], [130, ground_level - block_size * 3]),
            update_position(blockphoto[7], [150, ground_level - block_size * 3]),
            update_position(blockphoto[8], [170, ground_level - block_size * 3]),
            update_position(blockphoto[269], [190, ground_level - block_size * 3
```

```
}else if (level===2){
    handle_input2();

    update_position(fire_text,[140,525]);

        update_position(player0[0], [-100, -100]);
        update_position(player0[1], [-100, -100]);
        update_position(player0[2], [-100, -100]);
        update_position(player0[3], [-100, -100]);
        update_position(fullpath, [-100, -100]);


    if (check_collision3(player22, blockphoto[214])) {
        move_monster=1;

    }

    if (move_monster===1){
        if(move<380){
```

# Game Video



# Play it Online

>>>

# Thanks for your listening!